Compilação de Cabeçalhos de Regras do Snort 3 em Pré-filtros P4 Conservadores: Avaliação no BMv2

Arthur de Freitas Medeiros
Universidade Federal do Pampa (UNIPAMPA), Alegrete, RS, Brasil
arthurfreitas.aluno@unipampa.edu.br

Silvio E. Quincozes
Faculdade de Computação (FACOM), Universidade Federal de Uberlândia
Monte Carmelo, MG, Brasil
sequincozes@ufu.br

Outubro de 2026

Resumo

Planos de dados programáveis podem avaliar predicados de pacote antes de um IDS em software, mas as regras do Snort 3 dependem de payload, fluxo e serviço, que um alvo P4 não representa. Apresentamos um compilador Snort-para-P4 que classifica a relação de cada regra com seu predicado como EXACT, NECESSARY ou UNSUPPORTED e registra tudo o que omite. Em 2.931 regras baseadas em serviço, mantém só os endereços, pois o Snort 3 não checa porta e protocolo delas. Das 4.017 regras, 3.996 são necessárias e 21 não suportadas; o backend instala 235 entradas no BMv2. O pré-filtro global marca 100% dos pacotes elegíveis em três traces do CICIDS2017, os alertas observados são preservados e uma campanha com 1.000 regras, com o Snort como oráculo, não encontra violação. A proveniência reduz as regras candidatas em só 18,3% do ruleset, contra 81,8% numa compilação anterior que mantinha as portas.

Abstract

Programmable data planes can evaluate packet predicates before a software IDS, but Snort 3 rules also depend on payload, flow and service semantics that a P4 target cannot represent. We present a Snort-to-P4 compiler that classifies each rule's relation to its predicate as EXACT, NECESSARY or UNSUPPORTED and records every omitted component. For 2,931 service-based rules it keeps only the header addresses, since Snort 3 skips their ports and protocol. Of 4,017 rules, 3,996 are necessary and 21 unsupported; the backend installs 235 entries on BMv2. The global prefilter marks 100% of eligible packets in three CICIDS2017 traces, observed alerts are preserved, and a 1,000-rule campaign with Snort as oracle finds no violation. Provenance reduces the candidate rule set by only 18.3% of the ruleset, against 81.8% under an earlier compilation that kept the ports.

1 Introdução

Contexto. Este trabalho trata da tradução automática de regras de sistemas de detecção de intrusão (IDS) baseados em assinatura para switches programáveis. O IDS adotado é o Snort 3 [11,17], cujas regras combinam condições de cabeçalho (protocolo, endereços, portas, flags TCP) com condições que dependem do conteúdo do pacote, do estado do fluxo e do serviço de aplicação. A linguagem alvo é P4 [2], que descreve o processamento de pacotes no plano de dados e executa em velocidade de linha, mas oferece um modelo computacional restrito: casamento por tabelas sobre campos de cabeçalho, sem inspeção de payload arbitrária e sem estado por fluxo

no modelo usado. A ideia explorada é a pré-filtragem: deixar o switch decidir cedo quais pacotes precisam da inspeção completa do IDS em software.

Motivação. A inspeção em software é o gargalo reconhecido dessa arquitetura, e a literatura já demonstrou que o plano de dados programável pode aliviá-la. P4ID [7] deriva filtros diretamente de regras Snort e reporta redução no volume encaminhado ao IDS. O 4MIDable [8] reporta reduções no tráfego processado sem alterar as taxas de detecção observadas, em alvos BMv2 e Tofino. O P4-ONIDS [14] trata explicitamente das restrições de recursos do plano de dados por meio de agregação e deduplicação de regras. O conjunto desses resultados estabelece que a abordagem é viável e que há ganho a obter.

Justificativa. A fraqueza comum a esses trabalhos não está na viabilidade, e sim no que acontece com a parte da regra que não cabe no alvo. Uma regra Snort típica conjuga uma condição de cabeçalho com uma condição de payload ou de estado. Ao gerar o predicado P4, a condição que não cabe precisa de tratamento explícito, e, no escopo revisado, nenhum dos trabalhos analisados documenta, regra a regra, qual relação semântica sobrevive à tradução. Isso importa porque o erro é silencioso e assimétrico: se o predicado gerado ficar mais *restritivo* que a regra de origem, o pré-filtro descarta pacotes que a regra original detectaria, e a falha não aparece em nenhuma métrica de cobertura de tradução. Há ainda uma fonte de erro menos óbvia, que este trabalho documenta: o cabeçalho de uma regra não diz, sozinho, o que o IDS exige. No Snort 3, uma regra associada a um serviço de aplicação deixa de ter porta e protocolo checados quando o serviço do fluxo é identificado, e um predicado que os exija fica mais restritivo que a regra.

Objetivo. Este trabalho propõe um compilador Snort-para-P4 que torna explícitas as consequências semânticas da compilação parcial. A propriedade central que ele busca preservar é

$$\text{Snort}(r, p) \Rightarrow \text{P4_candidato}(r, p),$$

ou seja: se a regra r dispararia sobre o pacote p, o predicado gerado obrigatoriamente marca p como candidato. O predicado pode ser mais permissivo que a regra, nunca mais restritivo. Quando essa garantia não pode ser assegurada, a regra é rotulada como não suportada em vez de ser traduzida de forma aproximada, e pacotes que o parser do switch não consegue interpretar são mantidos como candidatos (*fail-open*). As contribuições específicas são:

- Classificar a relação entre cada regra Snort e o predicado P4 gerado em três categorias — Exact, Necessary e Unsupported — segundo a garantia efetivamente preservada;
- Registrar em um manifesto de compilação, predicado a predicado, o que foi preservado, delegado ao Snort, expandido ou tratado por fallback, mantendo a proveniência entrada-P4 → regras de origem através do lowering e da deduplicação;
- Implementar o backend completo, da regra até a instalação das entradas via P4Runtime em um alvo BMv2/v1model, com um domínio de pacotes imposto no plano de dados e validado por uma suíte controlada e pela contabilidade do replay;
- Avaliar separadamente a segurança semântica e a seletividade operacional, mostrando empiricamente que são propriedades independentes;
- Executar uma campanha diferencial fim a fim sobre 1.000 regras amostradas, com o Snort 3 como oráculo e o BMv2 como alvo;
- Mostrar, a partir do código-fonte do Snort 3.12.2.0, que regras *service-based* não tornam porta e protocolo do cabeçalho condições necessárias, compilar as 2.931 regras afetadas segundo essa semântica e medir no BMv2 o que uma compilação anterior, que os mantinha, deixava escapar;
- Explorar um uso alternativo da proveniência retida — em vez de uma decisão binária, derivar o conjunto de regras candidatas por pacote — e medir os limites dessa redução.

2 Fundamentação Teórica

Snort 3 e seu modelo de regras. O Snort 3 é um IDS de rede baseado em assinaturas. Cada regra tem um cabeçalho — ação, protocolo, endereço e porta de origem, direção, endereço e porta de destino — e um corpo de opções entre parênteses. As opções incluem tanto condições de detecção (`content`, `pcre`, `flow`, `flowbits`, `flags`) quanto metadados (`msg`, `sid`, `classtype`). A distinção relevante aqui é que só uma parte dessas condições é avaliável sobre campos de cabeçalho; o restante exige inspeção de payload ou estado acumulado de conexões anteriores.

Regras por porta e por serviço. O Snort 3 distingue regras *port-based* de regras *service-based*. Uma regra é service-based quando declara a opção `service`, usa uma opção específica de um serviço ou nomeia um serviço no cabeçalho, como em `alert http` [17]. Numa regra port-based, todo componente do cabeçalho precisa casar. Numa regra service-based, uma vez que o Snort identifica o serviço do fluxo como um dos serviços da regra, os endereços continuam sendo checados, mas as portas e o protocolo não (Seção 5.3). Um predicado de cabeçalho extraído da regra é, portanto, uma aproximação cuja relação com a regra precisa ser declarada, e não presumida.

P4 e o modelo v1model. P4 [15] é uma linguagem para descrever o processamento de pacotes no plano de dados. O programa define o parser de cabeçalhos, tabelas de casamento e ações. O modelo arquitetural usado aqui é o v1model. Uma tabela P4 casa campos extraídos do cabeçalho segundo tipos de casamento (`exact`, `lpm`, `ternary`, `range`) e dispara uma ação. O modelo não oferece varredura de payload nem estado por fluxo, o que estabelece o limite do que pode ser traduzido.

BMv2 e P4Runtime. O BMv2 [10] é o switch de referência em software usado como alvo de execução. As entradas das tabelas não fazem parte do programa P4: são instaladas em tempo de execução pelo plano de controle via P4Runtime [16], um protocolo gRPC que usa o arquivo P4Info gerado pelo compilador P4 para identificar tabelas, campos e ações por identificadores numéricos.

Pré-filtragem e predicados necessários. Um pré-filtro é uma condição avaliada antes do IDS que decide se o pacote precisa de inspeção. Diz-se que o predicado p_r é *necessário* para a regra r quando todo pacote que satisfaz r também satisfaz p_r. Um predicado necessário pode ser usado com segurança para pré-filtragem, porque nunca descarta um pacote que a regra detectaria — ainda que retenha tráfego adicional. É essa noção que fundamenta a propriedade preservada neste trabalho.

CICIDS2017. Conjunto de traços de tráfego produzido pelo Canadian Institute for Cybersecurity [13], com janelas rotuladas por tipo de ataque. As janelas de quinta-feira (SQL Injection, XSS e Brute Force) foram usadas na avaliação.

3 Trabalhos Relacionados

P4ID [7]. Deriva filtros de pacotes diretamente de regras Snort, usando endereços IP, portas de transporte, protocolos e flags TCP, e combina estágios de processamento sem estado e com estado. Sua principal contribuição é demonstrar que filtragem derivada de regras é viável no plano de dados programável e que o encaminhamento ao IDS pode ser reduzido. A limitação central, do ponto de vista deste trabalho, é que a extração de campos não vem acompanhada, no escopo revisado, da declaração da relação semântica entre a regra de origem e o filtro gerado: não fica registrado o que foi descartado ao converter uma regra que também dependia de payload.

P4-ONIDS [14]. Trata explicitamente das limitações de recursos de planos de dados restritos, aplicando representação de faixas, agregação de regras, deduplicação e uso de informação de fluxo para limitar os pacotes encaminhados ao NIDS. A contribuição é mostrar que a viabilidade em hardware real depende de compressão da representação. A limitação é que a agregação e a deduplicação são tratadas como otimizações de ocupação de tabela, sem que o efeito dessas

transformações sobre a garantia semântica de cada regra agregada seja explicitado.

4MIDable [8]. Propõe um arcabouço para integrar middleboxes de segurança existentes a uma infraestrutura P4/SDN. Seu agente P4ID suporta processamento com estado e realimentação para um IDS não modificado, e foi avaliado tanto em BMv2 quanto em Tofino, reportando redução de tráfego processado sem afetar as taxas de detecção observadas. A contribuição é a integração prática com IDS legado e a validação em hardware. A limitação é que a preservação da detecção é estabelecida empiricamente sobre os traços avaliados, e não por uma relação por regra que valha independentemente do tráfego observado.

P4NIPS [12]. Apresenta uma implementação recente de prevenção de intrusão em P4, estendendo a linha de detecção para ação de bloqueio no plano de dados. Contribui com uma realização concreta e atual do conceito. A limitação, para os fins deste trabalho, é a mesma das anteriores: o foco está na construção do sistema, não na caracterização do que a tradução preserva ou perde.

eRBF — Brum et al. [3]. Revisita a pré-filtragem baseada em regras considerando que o IDS pode precisar de tráfego que não satisfaz diretamente as condições extraídas da regra. A abordagem incorpora contexto de fluxo, incluindo estabelecimento e encerramento de conexões TCP e dependências de protocolo selecionadas, e é avaliada com Snort 3 sobre conjuntos grandes de regras. A contribuição é mostrar que pré-filtragem agressiva precisa levar em conta o contexto exigido pelo IDS — um argumento diretamente alinhado ao deste trabalho. A limitação é que a avaliação é feita em software sobre traços, sem tratar do mapeamento dessas semânticas para as restrições de processamento e de recursos de um alvo P4.

RNA [5]. Segue direção complementar, orientada a eventos: analisa eventos selecionados de scripts Zeek e os mapeia para templates de protocolo e de evento para dispositivos de encaminhamento programáveis, junto a mecanismos no host. A contribuição é demonstrar a geração coordenada de funcionalidade distribuída entre host e plano de dados. A limitação, quanto ao escopo aqui tratado, é que parte de eventos Zeek selecionados e de templates predefinidos, e não de uma classificação semântica geral de regras de IDS por assinatura segundo a garantia preservada.

Contexto mais amplo. Surveys sobre fundamentos e aplicações de P4 [6], sobre aplicações de segurança em switches P4 [1], sobre IDS em plano de dados programável [4] e trabalhos orientados à implementação [9] documentam a amplitude do campo e reforçam que o mapeamento de funcionalidade de segurança para alvos programáveis é limitado por recursos e por expressividade.

3.1 Posicionamento

A Tabela 1 compara os sistemas quanto à origem das regras, ao alvo avaliado, à existência de uma relação semântica declarada e à proveniência por regra. O padrão que emerge é consistente: os trabalhos demonstram viabilidade, integração e eficiência de representação, mas tratam a tradução parcial como um detalhe de implementação. Em nenhum deles, no escopo revisado, é explícita, por regra, a garantia que o predicado gerado preserva, nem a rastreabilidade da entrada instalada de volta às regras que a originaram. É essa lacuna que este trabalho endereça.

4 Proposta

A proposta é um compilador que traduz regras Snort 3 em entradas de tabela P4 buscando preservar a propriedade $\text{Snort}(r, p) \Rightarrow \text{P4_candidato}(r, p)$ e tornando explícito tudo aquilo que não pôde ser traduzido. O pipeline tem nove estágios:

1. **Parser** — lê a sintaxe da regra Snort 3 e produz uma representação estruturada, rejeitando as formas que o próprio Snort rejeitaria;
2. **Resolução de variáveis** — substitui variáveis de rede (`$HOME_NET`, `$EXTERNAL_NET`) e de serviço pelos valores do perfil de configuração;

Tabela 1: Comparação com os trabalhos relacionados. N.E.: não explícito no escopo revisado, o que não equivale a ausência do recurso.

Trabalho	Origem	Alvo	Semântica declarada	Proveniência por regra
P4ID	Snort	Plano P4	N.E.	N.E.
P4-ONIDS	NIDS	Plano P4	N.E.	N.E.
4MIDable	Agente P4ID	BMv2, Tofino	N.E.	N.E.
P4NIPS	N.E.	P4	N.E.	N.E.
eRBF	Snort 3	Software	N.E.	N.E.
RNA	Zeek	P4 + host	N.E.	N.E.
Este trabalho	**Snort 3**	**BMv2**	**Sim (3 classes)**	**Sim**

3. **IR semântica** — converte a regra em uma lista de predicados tipados (cabeçalho, payload, estado de fluxo, metadados);

4. **Análise de capacidade do alvo** — para cada predicado, decide se ele é representável na tabela P4 e atribui um tratamento: PRESERVED, EXPANDED, DELEGATED, FALLBACK ou OUT_OF_SCOPE;

5. **Classificador semântico** — a partir dos tratamentos, classifica a regra como EXACT (todo o significado cabe no P4), NECESSARY (o predicado é mais permissivo, e seguro) ou UNSUPPORTED (não há predicado seguro derivável);

6. **Manifesto** — registra a decisão de cada predicado de cada regra, com o motivo, e mantém a ligação entrada → regras de origem;

7. **Backend P4** — gera as entradas, expande listas de portas e regras bidirecionais, normaliza e deduplica;

8. **Lowering** — converte para valores e máscaras ternárias;

9. **Instalação** — gera o plano P4Runtime e instala no BMv2, verificando o contrato do P4Info e conferindo a codificação por round-trip.

O ponto de projeto que diferencia a proposta é o tratamento do caso em que a tradução não é possível. Em vez de descartar a condição que não cabe — o que tornaria o predicado mais restritivo e abriria um buraco de detecção —, a condição é delegada ao Snort e registrada no manifesto.

Domínio de pacotes imposto. A garantia pressupõe que o plano de dados leia os campos a que o predicado se refere: um parser que assumisse cabeçalho IPv4 fixo de 20 bytes leria bytes de opções como portas. O programa P4 define por isso um domínio suportado D: Ethernet sem VLAN com IPv4 direto, IHL = 5, sem fragmentação (flags nulas ou DF) e sem erro de parser. O parser só extrai a camada de transporte dentro de D, e o controle de ingresso repete a checagem antes de aplicar a tabela. Cada pacote incrementa exatamente um de três contadores: `P4_CANDIDATE` (alguma entrada casou dentro de D), `P4_NO_MATCH` (nenhuma casou dentro de D) e `PACKET_DOMAIN_FALLBACK` (fora de D, marcado como candidato incondicionalmente). O veredito *no-match*, o único que poderia justificar retirar um pacote do IDS, só é emitido dentro de D.

Cabeçalho de regras service-based. Para uma regra service-based, o compilador mantém só os endereços do cabeçalho e delega porta e protocolo ao Snort, mantendo o protocolo apenas quando a opção `flags` já restringe a regra a TCP, porque essa opção não casa outro protocolo (Seção 5.3). As regras que nomeiam o serviço no cabeçalho, como `alert http`, mantêm as portas do cabeçalho. Uma versão anterior do compilador tratava toda regra pelo cabeçalho inteiro; seus artefatos estão arquivados e se regeneram byte a byte.

5 Avaliação

A avaliação responde a quatro questões: (Q1) qual a cobertura da compilação semântica; (Q2) se o backend gerado é viável no alvo; (Q3) qual a seletividade do pré-filtro resultante; e (Q4) se os alertas observados são preservados e se a proveniência retida tem utilidade quando a seletividade binária é baixa. As três primeiras caracterizam a proposta; a última verifica empiricamente a garantia semântica. A Seção 5.3 compara os resultados com os da compilação anterior.

5.1 Metodologia

Conjunto de regras e perfil. O conjunto avaliado é o Snort 3 Community Ruleset, com 4.017 regras, sob o perfil experimental `cicids2017`, que resolve `$HOME_NET` para 192.168.10.0/24 e `$HTTP_PORTS` para 80 e 8080. As classificações dependem dessa configuração, e o perfil resolvido é registrado junto aos resultados.

Tráfego. Janelas de quinta-feira do CICIDS2017: SQL Injection (35.274 quadros) e XSS (191.848) como traços de desenvolvimento, e Brute Force (1.108.658) mantido como traço *held-out* — nenhum ajuste de política foi feito após inspecioná-lo.

Ambiente. Os alertas de referência vêm do Snort 3.12.2.0. O programa P4 é compilado pelo `p4c` 1.2.4.2, com o backend `p4c-bm2-ss`, e executado no BMv2 (`simple_switch_grpc` 1.15.0, modelo v1model) em Ubuntu 22.04, com as entradas instaladas via P4Runtime. O replay usa um socket AF_PACKET que transmite os bytes Ethernet capturados sem reserialização, a cada 2 ms (20 ms só com os pacotes de alerta) e com filas veth de 10.000 quadros, após instalação nova do pipeline, contadores conferidos em zero e uma auditoria de ociosidade com delta nulo; os quadros enviados são reconciliados com a soma dos três contadores.

Validação do domínio. Uma suíte controlada envia ao BMv2, com o plano global de 235 entradas instalado, um quadro por caso e compara o delta dos contadores com a expectativa offline. Os onze casos cobrem candidatos TCP, UDP e ICMP, um no-match garantido, flags TCP exatas e mascaradas, opções IPv4, primeiro fragmento com MF, fragmento não inicial, cabeçalho IPv4 truncado e cabeçalho TCP truncado.

Campanha diferencial fim a fim. Os traços só exercitam as poucas regras que disparam neles. Para testar a garantia sobre mais regras, um gerador sintetiza, a partir da representação intermediária de cada regra, um fluxo curto cuja testemunha deveria fazer o Snort disparar aquela regra: handshake TCP quando `flow` exige sessão estabelecida, cabeçalho com os valores da regra e cada `content` no buffer inspecionado. Cada testemunha tem um vizinho sem o conteúdo (negativo de payload) e, quando o plano da geração dos casos deixava espaço, um com porta de destino fora de todas as suas entradas (negativo de cabeçalho). Foi possível sintetizar testemunha para 2.142 das 4.017 regras; das 1.875 excluídas, 1.654 têm opções como `pcre`, `byte_test` ou `flowbits`, que não se satisfazem escolhendo bytes. Uma amostra de 1.000 regras, estratificada por protocolo, serviço e direção com semente fixa, gerou 6.684 quadros, todos dentro de D. O Snort lê o traço com o ruleset completo, o BMv2 recebe os quadros um a um, e cada caso recebe um veredito: Confirmado (o Snort alertou e todo quadro alertado foi candidato); Violação (o Snort alertou e a testemunha, ou outro quadro que satisfaz o cabeçalho da regra, foi no-match — o único resultado que contradiz a propriedade); Vácuo (o Snort não alertou, e o caso não testou nada); ou Dependência de estado (a testemunha foi candidata, mas o Snort atribuiu o alerta a um quadro da sessão que não satisfaz o cabeçalho e foi no-match). Regras em fallback não são pré-filtradas e são contadas à parte. O mesmo traço rodou contra o plano global e contra o plano do discriminador forte (Seção 5.2). Como cada testemunha usa uma porta do próprio cabeçalho, uma segunda campanha leva o fluxo HTTP de 200 regras `service:http` à porta 18080, que nenhum desses cabeçalhos lista (Seção 5.3).

Métricas. (i) cobertura de classificação semântica; (ii) número de entradas instaladas, sua validação e a redução do estado de instalação, $1 - N_{\text{install}}/N_{\text{source}}$; (iii) taxa de pacotes candidatos, calculada apenas sobre D; (iv) preservação de alertas, comparando a execução do Snort com

o ruleset completo contra a união das execuções com a partição candidata e com a partição UNSUPPORTED; e (v) conjunto de regras candidatas por pacote, $C(x) = \bigcup_{e \in E(x)} S(e)$, em que $E(x)$ são as entradas que casam o pacote x e $S(e)$ as regras de origem da entrada e. Sua redução é $R(x) = 1 - |C(x)|/B$ dentro da partição P4-elegível de B regras e $R_{\text{full}}(x) = 1 - (|C(x)| + F)/4017$ sobre o ruleset, em que F conta as regras de fallback, que valem para todo pacote.

Reprodutibilidade. O código, o perfil, os scripts de análise e os artefatos congelados estão em `https://github.com/medeiros08/Snort-p4`. Os dez artefatos do pipeline (manifesto semântico, partição de regras, entradas P4 e plano P4Runtime) são determinísticos: regenerados em uma cópia isolada a partir do código versionado, saem idênticos byte a byte, assim como os da compilação anterior. A suíte automatizada tem 27 módulos de teste; 24 passam e os outros 3, que exigem os bindings P4Runtime/gRPC, só executam no ambiente com o BMv2.

5.2 Resultados

Cobertura semântica (Q1). Das 4.017 regras, 3.996 foram classificadas como NECESSARY, 21 como UNSUPPORTED e **nenhuma** como EXACT. O parser leu 4.017 de 4.017 regras sem falha. No nível de predicado: 14.866 preservados, 29.411 delegados ao Snort, 323 expandidos, 65 em fallback e **zero** não avaliados — isto é, não houve predicado sobre o qual o compilador deixasse de decidir. O fato de EXACT ser zero é o resultado esperado, e não um defeito: nenhuma regra do conjunto tem todo o seu significado representável no plano de dados, porque praticamente todas combinam cabeçalho com `content` ou com estado de fluxo. As 21 UNSUPPORTED não são ignoradas: quatro têm cabeçalho não seletivo, dez são regras de serviço sem cabeçalho tradicional, sete são regras service-based sem restrição de endereço, cujas únicas condições seletivas eram porta e protocolo, e todas permanecem no Snort. Das 3.996 NECESSARY, 1.072 têm o predicado derivado do cabeçalho com as portas e 2.924, service-based, dos endereços (e do TCP, quando há `flags`).

Viabilidade do backend (Q2). As 4.040 instâncias de predicado geradas, expandidas e normalizadas resultaram em 231 casamentos únicos e, após o lowering de flags TCP, em 235 entradas de instalação — redução de 94,18% no *estado da tabela*, não no tráfego. A maior parte vem das regras service-based, cujos predicados só de endereço coincidem: uma única entrada, destino em `HOME_NET`, carrega a proveniência de 2.043 regras. As 235 entradas foram codificadas, instaladas no BMv2 e lidas de volta sem erro, dentro da capacidade de 8.192 entradas da tabela.

Domínio de pacotes. A suíte controlada passou nos onze casos com deltas exatos; os cinco fragmentados ou truncados caíram em `PACKET_DOMAIN_FALLBACK`. Nos traços, ficaram fora de D 350, 2.772 e 7.199 quadros, quase todos ARP e IPv6, e todos mantidos como candidatos. O replay de cada traço inteiro, nos dois planos, contou todos os quadros enviados, com cada categoria igual à do classificador offline.

Seletividade: o resultado negativo (Q3). O pré-filtro binário global marcou **100% dos pacotes elegíveis** como candidatos nos três traços (Tabela 2). A análise causal identificou entradas amplas: `p4-88` casa qualquer TCP e subsume 162 das 235 entradas, e `p4-10` e `p4-51` casam destino e origem em `HOME_NET` e carregam a proveniência de 2.043 e 900 regras, quase todas service-based. A ablação com discriminador forte considera forte a entrada que restringe porta, flags TCP ou prefixo IPv4 diferente de exatamente `HOME_NET`, e mantém uma regra P4-elegível só se todas as suas entradas forem fortes. Ela removeu dez entradas, manteve 225 para 739 regras elegíveis e pôs 3.257 em fallback operacional, além das 21 UNSUPPORTED; mesmo assim, no BMv2, marcou 92,173–92,298% dos pacotes nos traços de desenvolvimento e 94,885% no held-out. Quatro entradas retidas chegam a 90% dos pacotes de SQLi e XSS: uma casa TCP para `HOME_NET` com o bit ACK, vinda de uma única regra SMTP (SID 309, `flags:A+`), e três casam tráfego de `HOME_NET` para as portas 443, 80 e 53. São discriminadores fortes pelo critério estrutural, mas coincidem com o tráfego dominante.

Preservação de alertas (Q4). Nos três traços, a partição semântica não produziu alertas faltantes nem adicionais; as sete regras que a compilação atual acrescenta às UNSUPPORTED

Tabela 2: Seletividade e preservação por traço, medidas no BMv2 com contabilidade exata. Candidatos sobre os pacotes em D. Alertas do Snort com o ruleset completo; P4 cand.: pacotes de alerta marcados como candidatos em cada plano. No plano forte, todo alerta vem de regra em fallback.

Traço	Em D	Global cand.	Forte cand.	Alertas	SIDs	P4 cand. global	P4 cand. forte
SQLi	34.924	100%	92,298%	150	1	150	0
XSS	189.076	100%	92,173%	528	2	528	0
Brute Force	1.101.459	100%	94,885%	2.391	5	2.391	3

Tabela 3: Campanha diferencial sobre a mesma amostra de 1.000 regras, no mesmo traço. Fallb.: regras que o plano não pré-filtra. "Só própria": confirmadas cuja testemunha casou apenas entradas da própria regra.

Plano	Entr.	Conf.	Viol.	Dep. estado	Fallb.	Vácuo	Só própria
Global	235	958	0	0	2	40	0
Forte	225	134	0	0	826	40	36

não alertaram e não participam de dependência de flowbits, de modo que a medição feita com a partição anterior vale sem mudança. No BMv2, reproduzindo só os pacotes de alerta, todos foram candidatos no plano global (Tabela 2). No plano forte, todos menos três foram no-match, sem perda: todo alerta observado vem de regra em fallback operacional, que o Snort inspeciona sem o pré-filtro. A SID 254, responsável por 2.960 dos 3.069 alertas, declara `service:dns` e, sem UDP e porta 53, fica só com o destino `HOME_NET`, que não é discriminador forte. Nenhum alerta observado pertence a regra P4-elegível, e nenhum testa os conjuntos candidatos da Tabela 4. Registra-se ainda que nenhuma das 21 regras UNSUPPORTED disparou e que os traços acionaram apenas 1, 2 e 5 SIDs distintos, o que limita o poder de discriminação dessa verificação. Os nomes das janelas também não implicam que os alertas observados detectaram os ataques rotulados.

Campanha fim a fim (Q4). Nenhum caso resultou em VIOLAÇÃO em nenhum dos dois planos (Tabela 3), nenhum quadro caiu fora de D, e as duas atribuições de alertas a casos, por faixa de pacotes e por timestamp, concordaram em todos. No plano global, 958 regras foram confirmadas, 40 foram vácuas e duas, Kerberos (SIDs 56583 e 56584), ficaram em fallback por serem UNSUPPORTED; as 1.000 testemunhas e os 1.000 negativos de payload foram candidatos, e os 101 negativos de cabeçalho foram no-match. Nenhuma confirmação desse plano, porém, pode ser atribuída só ao predicado da própria regra: toda testemunha confirmada também casou uma entrada de outra regra. No plano do discriminador forte, 134 regras foram confirmadas, 36 delas somente pelas próprias entradas, de modo que um defeito de lowering nessas regras teria aparecido como violação; das 826 em fallback operacional, 78 tiveram a testemunha marcada no-match, o que não viola nada numa regra que o plano não pré-filtra. Nas duas rodadas, a categoria prevista pelo casador offline coincidiu com o contador do BMv2 em todos os 2.101 quadros de testemunhas e vizinhos. Em 606 regras HTTP em fallback, o Snort atribuiu o alerta ao ACK que dispara a inspeção da mensagem armazenada, e o plano forte marcou 445 desses ACKs como no-match: um sistema que retirasse do IDS os pacotes no-match dependeria de manter essas regras fora do pré-filtro, como faz o fallback.

Redução do espaço de regras candidatas (Q4). Usando a proveniência retida em vez da decisão binária, o conjunto médio de regras candidatas por pacote, dentro da partição P4-elegível de 739 regras, ficou em 3,97 (SQLi), 3,94 (XSS) e 4,48 (Brute Force *held-out*), redução de 99,39–99,47% (Tabela 4). As 3.278 regras de fallback (3.257 operacionais e 21 UNSUPPORTED) valem para todo pacote; contadas contra o ruleset completo, as reduções são de 18,30%, 18,30% e 18,29%, logo abaixo do teto de 18,40%. Agrupar regras também custa: por protocolo, com um grupo TCP de 642 regras, a redução dentro da partição cai para 20,10–25,38%; protocolo e porta da entrada chegam a 98,80–98,96%.

Tabela 4: Conjunto de regras candidatas por pacote (offline, plano do discriminador forte). R sobre as 739 regras P4-elegíveis; R_{full} sobre as 4.017, contando as 3.278 de fallback; Prot.: redução quando se encaminham grupos inteiros por protocolo.

Traço	Média	Mediana	Máx.	R	R_{full}	Prot.
SQLi	3,97	3	43	99,46%	18,30%	25,38%
XSS	3,94	3	43	99,47%	18,30%	23,95%
Brute Force	4,48	3	293	99,39%	18,29%	20,10%

Tabela 5: Compilação anterior (cabeçalho inteiro) e atual (só endereços nas regras service-based). Valores de traço no Brute Force held-out.

	Anterior	Atual
NECESSARY / UNSUPPORTED	4.003 / 14	3.996 / 21
Entradas (global / forte)	331 / 322	235 / 225
Regras elegíveis no forte	3.611	739
Candidatos no forte	98,942%	94,885%
R_{full}	81,81%	18,29%
Fora da porta: violações / fallback	190 / 0	0 / 190

5.3 Regras service-based e a compilação anterior

A premissa de que uma regra só dispara quando todo o seu cabeçalho casa vale para o Snort 3 apenas em parte. No Snort 3.12.2.0, antes de checar o cabeçalho de uma regra com serviços, o Snort compara o serviço identificado no fluxo com os da regra (`src/detection/detection_options.cc` e `src/detection/rtn_checks.cc`). Se o serviço é um deles, as checagens de porta são puladas, a de protocolo passa incondicionalmente, e só os endereços são checados; se é outro, a regra é rejeitada; só sem serviço identificado o cabeçalho inteiro é avaliado. Como a identificação do serviço pode vir da inspeção do conteúdo, uma requisição HTTP numa porta fora de `$HTTP_PORTS` ainda é avaliada pelas regras HTTP. A exceção são as regras que nomeiam o serviço no cabeçalho, como `alert http`, que o Snort marca como regras de usuário e que mantêm as portas checadas (`src/parser/parse_rule.cc`).

A versão anterior do compilador tratava o serviço como mais uma condição delegada e mantinha porta e protocolo no predicado, o que seria correto se o Snort exigisse serviço *e* cabeçalho. Isso afetava 2.931 regras: 2.928 declaram `service` (`http` em 2.314, `imap` em 150, `pop3` em 138, `smtp` em 131), e três (SIDs 529, 3238 e 3239) recebem `dcerpc` e `netbios-ssn` das opções `dce_iface` e `dce_stub_data` (`src/detection/fp_utils.cc`). Todas tinham protocolo no predicado, e 2.877 também porta. O plano global anterior, de 331 entradas, ainda cobria quase todas por entradas amplas sem porta, mas o plano do discriminador forte as removia. O teste com 200 regras `service:http` na porta 18080 exercita exatamente esse caso: o Snort alertou em 190, e o plano forte anterior marcou no-match as 190 testemunhas e o ACK ao qual o Snort atribui cada alerta, isto é, 190 violações. Com a compilação atual, o plano global marcou as 190 testemunhas como candidatas, e o plano forte tem as 190 regras em fallback operacional (Tabela 5).

A correção custa sobretudo seletividade: só sete regras perdem cobertura. Sem porta e protocolo, a maioria das regras service-based fica só com um endereço em `HOME_NET`, que o discriminador forte não considera discriminante, e a partição elegível cai de 3.611 para 739 regras. A redução do conjunto candidato sobre o ruleset, que na compilação anterior chegava a 81,81% no held-out, vinha em grande parte de portas que o Snort 3 não checa depois de identificar o serviço.

5.4 Discussão dos resultados

Os achados centrais são três. O primeiro é negativo: segurança semântica não implica seletividade operacional, e a mesma união de predicados que elimina a seletividade também elimina o poder de teste — no plano global, nenhuma confirmação da campanha dependia só da regra testada. O segundo é que a garantia de um predicado derivado de regra depende de como o IDS avalia a regra, e não só do texto do cabeçalho: a compilação que mantinha as portas deixava escapar alertas no plano forte, e sua aparente seletividade vinha em boa parte dessas portas. O terceiro é construtivo, mas limitado: a proveniência indica quais regras importam para cada pacote, porém, com a semântica do Snort 3, poucas regras restam discriminadas pelo cabeçalho, e a redução sobre o ruleset fica em 18,3%. Um seletor de candidatas que usasse também o serviço identificado — que o próprio Snort usa para rejeitar regras de outros serviços — é o passo natural seguinte e não foi avaliado aqui. Todas as medidas de conjunto candidato são de viabilidade *offline*: não foi implementada execução do Snort orientada por conjunto de candidatos.

5.5 Limitações

Os resultados valem para um ruleset, um perfil de variáveis e um conjunto de dados antigo em relação ao tráfego atual. Para as quantidades por pacote, medimos também, offline, a quinta-feira inteira (9.322.025 quadros, 9.240.405 em D): o pré-filtro global continua em 100%, o discriminador forte marca 95,335%, e a redução média é de 99,42% na partição elegível e 18,29% sobre o ruleset; as janelas não são atípicas do dia, mas os alertas do Snort não foram calculados para o restante dele. A campanha amplia a cobertura de regras para 958 e 134 confirmadas, mas com tráfego sintético, um ponto por regra e sempre numa porta do próprio cabeçalho, de modo que só o teste dedicado observa a avaliação service-based; as 1.875 regras sem testemunha sintetizável não foram testadas. A semântica service-based foi estabelecida pela leitura do código-fonte e da documentação do Snort 3.12.2.0 e confirmada num teste de plano de dados só para um serviço e uma porta. A partição do discriminador forte não é fechada sob flowbits: 24 arestas a cruzam, todas no flowbit `smb.trans2`, e uma execução que separasse os dois lados teria de manter esse componente junto. O alvo é o BMv2 em software: não houve medição de vazão, latência ou recursos de hardware. Fora do ruleset avaliado, o compilador só foi exercitado na compilação, sobre as regras do Snort 2.9.15.1 distribuídas pelo Ubuntu; as dez regras só de cabeçalho desse ruleset, únicas classificadas como EXACT, concordaram com o Snort nos 340 pares (quadro, regra) amostrados. Por fim, algumas condições de cabeçalho representáveis ainda são omitidas, o que só alarga os predicados: faixas de porta, tipo e código ICMP e `ip_proto`. Representá-las elevaria a redução sobre o ruleset do plano forte em no máximo 1,3, 3,1 e 0,07 ponto percentual, respectivamente.

6 Conclusão e Trabalhos Futuros

Sistemas de detecção de intrusão baseados em assinatura enfrentam custo crescente de inspeção em software, e a pré-filtragem em switches programáveis é a resposta natural. A literatura demonstrou a viabilidade dessa abordagem, mas tratou a tradução parcial de regras como detalhe de implementação: quando uma condição de payload ou de estado não cabe no plano de dados, o que foi feito com ela não é explicitado. Isso é um problema mal resolvido porque o erro resultante é silencioso — um predicado mais restritivo que a regra de origem descarta tráfego que deveria ser inspecionado, sem que nenhuma métrica de cobertura de tradução o revele.

Este trabalho desenvolveu um compilador Snort-para-P4 que busca garantir que todo pacote sobre o qual uma regra dispararia seja marcado como candidato, e que registra explicitamente cada componente semântico omitido. Das 4.017 regras avaliadas, 3.996 resultaram em predicados necessários e 21 foram declaradas não suportadas; o backend instalou 235 entradas em um alvo BMv2, com fail-open fora do domínio que o parser interpreta. Os alertas observados

foram preservados, e a campanha fim a fim sobre 1.000 regras não encontrou violação. O principal resultado é negativo e informativo: o pré-filtro binário marcou 100% dos pacotes como candidatos. A proveniência retida reduz o conjunto de regras candidatas a cerca de quatro das 739 elegíveis, mas só 18,3% sobre o ruleset; a compilação anterior, que mantinha portas que o Snort 3 ignora em 2.931 regras service-based, prometia 81,8% e deixava escapar alertas. A contribuição para a literatura é a separação explícita entre segurança semântica e seletividade operacional, acompanhada de um manifesto de compilação que torna auditável o que cada tradução preserva, e a constatação de que essa garantia, e a seletividade aparente, dependem de como o IDS avalia a regra, e não apenas do seu texto.

Como trabalhos futuros, três direções se destacam. A primeira é um seletor de regras candidatas que use o serviço identificado, além do cabeçalho, e a execução do Snort orientada por esse conjunto, transformando a redução medida *offline* em ganho efetivo — o que exige tratar as dependências de estado entre regras, como as dos ACKs HTTP e dos flowbits da campanha. A segunda é ampliar a expressividade do backend, incorporando faixas de porta por decomposição ternária e campos ICMP à chave da tabela. A terceira é avaliar o custo de execução em alvos de hardware e sobre conjuntos de regras, perfis e tráfegos mais amplos, incluindo medição de vazão e latência, que não foram objeto desta avaliação.

Referências

[1] Ali AlSabeh, Joseph Khoury, Elie Kfoury, Jorge Crichigno, and Elias Bou-Harb. A survey on security applications of P4 programmable switches and a STRIDE-based vulnerability assessment. *Computer Networks*, 207:108800, 2022.

[2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3):87–95, 2014.

[3] H. B. Brum, L. A. D. Knob, T. Ferreto, and D. Siracusa. Rethinking NIDS rule-based pre-filtering. In *IFIP/IEEE International Symposium on Integrated Network Management and Network Operations (CNSM)*, 2025.

[4] Mah-Rukh Fida and Azza H. Ahmed. Leveraging programmable data plane for network intrusion detection: A survey. *Computer Networks*, 281:112202, 2026.

[5] L. S. Hagen, A. S. Ilha, R. Parizotto, and L. P. Gaspary. RNA: Automating IDS/IPS event detection offload into programmable forwarding devices. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2025.

[6] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. A survey on data plane programming with P4: Fundamentals, advances, and applied research. *Journal of Network and Computer Applications*, 212:103561, 2023.

[7] Benjamin Lewis, Matthew Broadbent, and Nicholas Race. P4ID: P4 enhanced intrusion detection. In *2019 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2019.

[8] Benjamin Lewis, Matthew Broadbent, Charalampos Rotsos, and Nicholas Race. 4MIDable: Flexible network offloading for security VNFs. *Journal of Network and Systems Management*, 31:52, 2023.

[9] Ali Mazloum, Ali AlSabeh, Elie Kfoury, and Jorge Crichigno. Security applications in P4: Implementation and lessons learned. *Computer Networks*, 257:111011, 2025.

[10] p4lang. BMv2: The behavioral model software switch. `https://github.com/p4lang/behavioral-model`, 2025. `simple_switch_grpc` 1.15.0, commit 68f4a978.

[11] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, WA, USA, 1999. USENIX Association.

[12] Eduardo Vudala Senoski, João Ribeiro Andreotii, Raphael Kaviak Machnicki, Jorge Pires Correia, and Vinicius Fulber-Garcia. P4NIPS: P4 network intrusion prevention system. In *Anais Estendidos do Simpósio Brasileiro de Cibersegurança (SBSeg)*, pages 75–84, 2025.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[14] Kairo Tavares and Tiago Ferreto. P4-ONIDS: A P4-based NIDS optimized for constrained programmable data planes in SDN. In *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 434–447, 2021.

[15] The P4 Language Consortium. *$P4_{16}$ Language Specification*, 2023. Version 1.2.4, `https://p4.org/specs/`.

[16] The P4.org API Working Group. *P4Runtime Specification*, 2024. Version 1.4.1, `https://p4.org/specs/`.

[17] The Snort Team. *Snort 3 User Manual*, 2026. Snort 3.12.2.0, `https://github.com/snort3/snort3`.